Lab 01 — Volunteer Matching Portal

Connect local organizations that need help with students who have time and skills to give.

Introduction

This is the one project of the three that you build **for other people**. Nobody at your table is the user. That makes it harder, and more interesting: you have to think about two groups at once — the organizations posting needs, and the students answering them.

Before You Start

- **One laptop, five people.** Decide who drives. The driver types; everyone else plans, watches the output and proposes what to try next. Swap if you like.
- **Your table's job is not to finish.** It's to get something real running and understand how you got there. Nobody expects a finished product in an hour.
- **Laptops stay closed** until Step 02. The first thirty minutes are paper and pens. That isn't a warm-up exercise — it's the part that determines whether the build works.

What You'll Build

A working web page where:

- An organization can post something it needs help with.
- A student can find the needs that match their time and skills.
- The two get matched — and **something happens** as a result.

That last one is the interesting part, and Step 01 is where you decide what it means.

Step 01: Brainstorm Together — Laptops Closed

About 30 minutes. Paper and pens on the table.

Write your answers down. At the end of this step you'll hand what you wrote to Claude, so write it for a reader, not as private shorthand.

Who Is This For?

- Name the two sides. Who posts a need? Who answers it?
- Pick one real example of each. A specific organization, a specific student. Invent them, but make them specific.
- What does each one want in the ten seconds they look at your page?

What Does the Screen Show?

- Sketch it. One page, boxes and labels, no artistry required.
- What is the **first** thing a visitor sees? What can they do without typing anything?
- What do you deliberately leave out? Name two things you're not building.

Where Does the Data Come From?

- You will be given a set of volunteer opportunities to work with.

[PLACEHOLDER - how students connect to the workshop MCP server, once the server exists]

- What fields does each opportunity have? Which ones does your page actually use?

What Does It Do, Not Just Show?

This is the agentic question, and the one worth most of your time.

- A page that **displays** information is a brochure. What does yours **do**?
- Some directions worth arguing about:
    - It notices a new need that matches a student's skills and tells them.
    - It drafts the introduction email between the two sides.
    - It runs each morning and sends a short digest of what changed.
    - It flags opportunities that are about to expire.
- Pick **one**. Write down what triggers it, and what the result is.

Step 02: Turn Your Notes into a Prompt

Now the laptop opens.

You have a page of decisions. Claude needs them as instructions.

- **Open a Cowork session** connected to your project folder from Lab 00.
- **Two ways to get your plan in**, and the second is usually better:
  - Type it out.
  - **Photograph your notes** with a phone and attach the picture. Claude reads handwriting and sketches. Your sketch of the screen is often clearer than a paragraph describing it.
- Write the prompt in your own words. It should say who it's for, what the screen shows, where the data comes from, and what it does. Don't worry about sounding technical.

Step 03: Let Claude Interview You

Optional, and recommended. Five minutes that saves twenty.

Before asking Claude to build anything, ask it to ask **you** questions:

> **PROMPT**
>
> `Before you build anything, interview me. Ask me five multiple-choice questions about what I want, one at a time. Then write a short plan as MVP Plan.md and show me a simple mockup with sample data. Wait for me to approve it before you write any other code.`

This catches the thing your table forgot. It is also, in miniature, the whole point of the lecture: you are directing an agent, not placing an order.

Step 04: Build It

- Send your prompt.
- Claude will ask permission before it does things. **Click Allow** — or **Allow for this task** to reduce the clicking.

Step 05: While You Wait

Builds take **ten minutes or more**. Plan for it.

- **Stay near the laptop.** Claude pauses to ask questions and waits until someone answers. Nothing moves while it waits.
- **Non-drivers, you have work to do.** Discuss:
  - *Why is Markdown a useful language for configuring agents?*
  - What would you have to change to make this work for a different city?
  - What could go wrong if this were real and had real users?
- **Compare with your neighbors.** The table next to you made different choices.

Step 06: Run It and Try It

- Claude will tell you how to start your app and give you a web address to open.
- Try it as each of your two users. Does the organization's side make sense? The student's?
- Find one thing that's wrong or missing. That's your next prompt.

Step 07: Add Your Own Twist

Whatever time is left is yours. Change something that makes it yours rather than ours.

If You Get Stuck

- **Strange file errors, or the build keeps failing:** check your project folder isn't in iCloud, Dropbox, OneDrive, Box or Google Drive. Move it to your home folder and try again.
- **Claude seems frozen:** look for a permission prompt waiting for a click.
- **The app won't start:** paste the error message back to Claude and ask it to fix it. This is normal and expected.

- **You're out of ideas:** see the Answer Sheet below.

Key Takeaways

- The thirty minutes on paper did more for your build than any single prompt.
- You directed an agent. It planned, used tools and acted — you decided what mattered.
- The gap between "displays information" and "does something" is the gap between an app and an agent.

Answer Sheet

Use these if your table is stuck. Reading them first spoils the exercise.

[PLACEHOLDER - suggested prompts: a starting prompt, a prompt for adding the agentic action, and two recovery prompts for common failures]